

1339. Maximum Product of Splitted Binary Tree

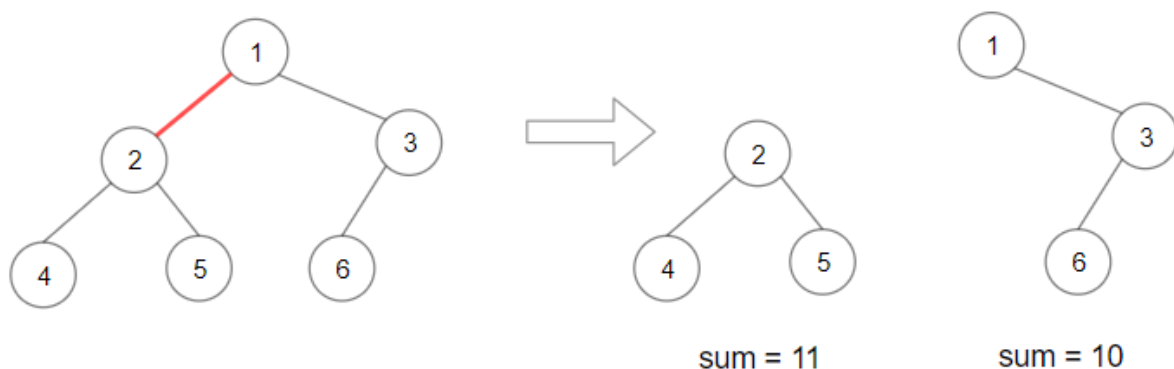
Statement:-

Given the root of a binary tree, split the binary tree into two subtrees by removing one edge such that the product of the sums of the subtrees is maximized.

Return *the maximum product of the sums of the two subtrees*. Since the answer may be too large, return it **modulo** $10^9 + 7$.

Note that you need to maximize the answer before taking the mod and not after taking it.

Example 1:

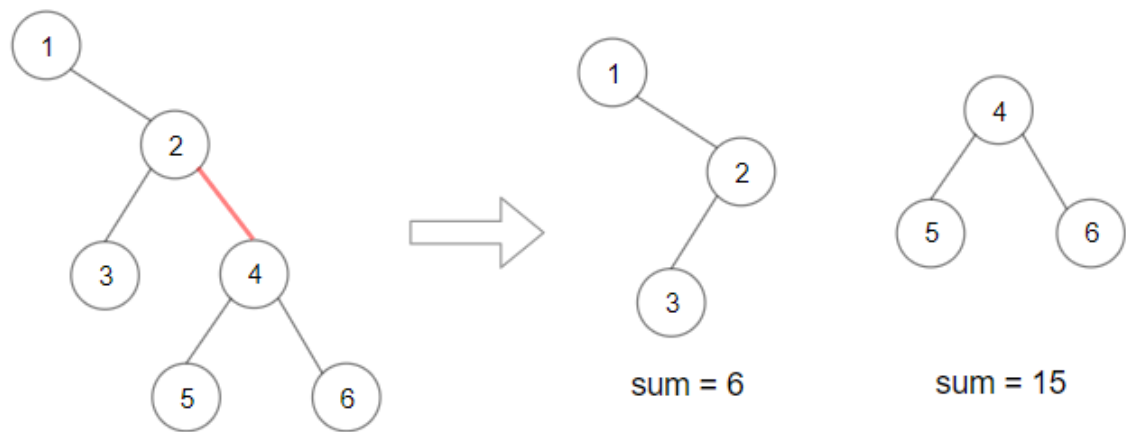


Input: root = [1,2,3,4,5,6]

Output: 110

Explanation: Remove the red edge and get 2 binary trees with sum 11 and 10. Their product is 110 (11×10)

Example 2:



Input: root = [1,null,2,3,4,null,null,5,6]

Output: 90

Explanation: Remove the red edge and get 2 binary trees with sum 15 and 6. Their product is 90 (15×6)

Constraints:

- The number of nodes in the tree is in the range $[2, 5 \times 10^4]$.
- $1 \leq \text{Node.val} \leq 10^4$

Explanation in Hindi

हमें एक **Binary Tree** दिया गया है।

हमें **सिर्फ 1 edge (connection)** हटानी है, जिससे tree **2 subtrees** में टूट जाए।

फिर हमें:

- दोनों subtrees का **sum** निकालना है
- उन दोनों sums का **product** निकालना है
- ऐसा edge चुनना है जिससे **product maximum** हो

👉 अंत में answer को $(10^9 + 7)$ से modulo करना है

⚠ लेकिन **maximum product पहले निकालना है**, modulo बाद में

◆ Main Idea (Core Concept)

अगर:

- पूरे tree का sum = **TOTAL**
- किसी subtree का sum = **S**

तो दूसरी subtree का sum होगा:

$TOTAL - S$

👉 उस edge को काटने पर product होगा:

$S \times (TOTAL - S)$

हमें हर possible subtree के लिए यह product निकालकर **maximum** ढूँढना है।

◆ Solution Strategy (2 DFS Approach)

✅ Step 1: Total Sum निकालो

पूरे tree का sum निकालो (DFS से)

✓ **Step 2: हर Subtree का Sum निकालो**

DFS करते हुए:

- हर node को root मानकर subtree sum निकालो
- $\text{product} = \text{subtreeSum} \times (\text{totalSum} - \text{subtreeSum})$
- maxProduct update करो

◆ **Example 1 Explanation (Hindi)**

Input:

root = [1,2,3,4,5,6]

Step 1: Total Sum

$1+2+3+4+5+6 = 21$

Step 2: Best Edge Cut

अगर हम 1-3 वाली edge हटाते हैं:

- Left subtree sum = 11 (2+4+5)
- Right subtree sum = 10 (1+3+6)

Product:

$11 \times 10 = 110$ ✓ (maximum)

Output:

110

◆ **Example 2 Explanation (Hindi)**

Input:

root = [1,null,2,3,4,null,null,5,6]

Total Sum:

$1+2+3+4+5+6 = 21$

Best Cut:

- Subtree 1 sum = 15
- Subtree 2 sum = 6

Product:

$15 \times 6 = 90$

Output:

90

◆ Flowchart (Text Form – Exam Friendly)

START

|

v

Read Binary Tree

|

v

DFS → Calculate TOTAL_SUM

|

v

DFS again:

|

```

└─ For each node:
|   └─ Find subtreeSum
|   └─ product = subtreeSum × (TOTAL_SUM – subtreeSum)
|   └─ Update maxProduct
|
v
Return maxProduct % (10^9 + 7)
|
v
END

```

◆ Important Points (Must Remember)

- Edge हटाने का मतलब = **subtree root चुनना**
 - Root node को नहीं काटते (क्योंकि parent नहीं होता)
 - Max निकालना है **modulo से पहले**
 - Time Complexity: **O(N)**
 - Space Complexity: **O(N)** (DFS stack)
-

◆ One-Line Summary (For Exam)

Binary Tree में हर subtree का sum निकालकर
 $\text{subtreeSum} \times (\text{totalSum} - \text{subtreeSum})$
 का maximum product निकाला जाता है।

Code in Java

```
class Solution {  
    long MOD = 1000000007L;  
    long ans = 0;  
  
    private long dfs(TreeNode node) {  
        if (node == null) return 0;  
  
        node.val += dfs(node.left) + dfs(node.right);  
        return node.val;  
    }  
  
    public int maxProduct(TreeNode root) {  
        long total = dfs(root);  
  
        Queue q = new LinkedList<>();  
        q.add(root);  
  
        while (!q.isEmpty()) {  
            TreeNode node = q.poll();  
            if (node == null) continue;  
  
            long cur = (total - node.val) * node.val;  
            ans = Math.max(ans, cur);  
  
            if (node.left != null) q.add(node.left);  
            if (node.right != null) q.add(node.right);  
        }  
  
        return (int)(ans % MOD);  
    }  
}
```

